Московский Физико–Технический Институт

Численные методы
Задание №7

ФИО: Лазарь Владислав Игоревич
Группа: Б01-207а
Почта: lazar.v@phystech.edu

Долгопрудный, 2026 г.

Введение

Задача реализации натурального логарифма $\ln(x)$ для типа `float` требует обеспечения точности в пределах `3.5 ULP` в диапазоне $2^{-126} \leqslant x < 2^{127}$. Основная сложность заключается в минимизации накопленной ошибки округления при работе с экспонентой и мантиссой числа в формате `IEEE 754`.

Описание решения

Алгоритм будет основан на классических шагах:

1. Фильтрация — проверка аргумента на попадание в область определения. Также обработка специальных значений (`NaN`, $\pm$`inf`).
2. Редукция аргумента — приведение аргумента из широкого диапазона к малому $\big($например, от $[-\infty,\ +\infty]$ к $[0,\ k)\big)$, где $k \sim 10^0$.
3. Аппроксимация — вычисление значения на интервале $[0,\ k)$ с помощью полиномов и табличных данных.
4. Реконструкция — обратное преобразование результата с учетом проведенной редукции.

1. Фильтрация

1.1. Реинтерпретация бит:

Процессор воспринимает `float` как структуру из знакового бита, экспоненты и мантиссы. Я использовал `union`, чтобы посмотреть на эти же 32 бита как на целое число `uint32_t` для манипулирования битами напрямую:

```
typedef union {
  float f;
  uint32_t i;
} float_conv;
```

Теперь можем легко получить `ix` и `abs_ix` из изначального `float x`. Здесь используется битовая маска

$$\texttt{0x7FFFFFFF} \equiv \texttt{0111 1111 1111 1111 1111 1111 1111 1111}.$$

```
conv.f = x;
uint32_t ix = conv.i;
uint32_t abs_ix = ix & 0x7FFFFFFF;
```

1.2. Проверка на отрицательные числа и нуль:

Нужно обработать левую границу области определения логарифма, так как $\ln(x)$ определен только для $x > 0$. Здесь используется битовая маска

$$\texttt{0x80000000} \equiv \texttt{1000 0000 0000 0000 0000 0000 0000 0000}.$$

Если же `ix >= 0x80000000`, это значит, что `x` отрицательный. Далее, в зависимости от остальных бит, `x` может быть либо `-infinity` (если все биты кроме знакового выставлены в 0), либо `NaN` (в остальных случаях).

```
if (abs_ix == 0) { // Если x = +0.0 или x = -0.0
  errno = ERANGE;
  feraiseexcept(FE_DIVBYZERO);
  return -1.0f / 0.0f;
}

if (ix >= 0x80000000) { // Если x <= 0
  errno = EDOM;
  feraiseexcept(FE_INVALID);
  return (x - x) / 0.0f; // NaN
}
```

1.3. Проверка на `NaN` и `+infinity`:

Здесь используется битовая маска

$$\texttt{0x7f800000} \equiv \texttt{0 11111111 00000000000000000000000}.$$

Все биты экспоненты заполнены единицами, то есть `0x7f800000` соответствует `+infinity`. Любое число больше этого (в беззнаковом виде) — это `NaN`.

```
if (ix >= 0x7f800000) // NaN или +infinity
  return x;
```

1.4. Нормализация субнормальных чисел:

В стандарте `IEEE 754` субнормальные числа — это числа, которые настолько близки к нулю, что их экспонента равна нулю, а мантисса не имеет неявной единицы в начале.

Здесь используется битовая маска

$$\texttt{0x00800000} \equiv \texttt{0 00000001 00000000000000000000000}.$$

У субнормальных чисел экспонента как бы "спрятана" внутри мантиссы (число хранится как $0.f \cdot 2^{-126}$ вместо $1.f \cdot 2^{k}$). Чтобы снизить ошибку, нужно учесть наличие субнормальных чисел, нормализовав их — то есть превратив в обычные, сдвинув биты мантиссы влево до появления первой единицы.

```
int subnormal_bits = 0;
if (ix < 0x00800000) {      // Если x - субнормальное число
    x *= 33554432.0f;       // Умножаем на 2^25
    conv.f = x;             // Записываем результат обратно в union
    ix = conv.i;            // Снова берем битовое представление
    subnormal_bits -= 25;   // Запоминаем, что искусственно увеличили число
}
```

Использование сдвига 2^{25} (вместо минимально необходимых 2^{23}) обеспечивает гарантированный переход числа из субнормальной области в нормализованную с достаточным запасом по величине экспоненты для корректной последующей экстракции значащих бит.

2. Редукция аргумента

2.1. Извлечение реальной экспоненты:

По стандарту `IEEE 754` экспонента хранится в битах с 23 по 30 в смещенном (`bias`) виде.

```
subnormal_bits += (int)(ix >> 23) - 127;
```

Здесь было сделано следующее:

1. Биты были сдвинуты на 23 вправо, чтобы оставить только экспоненту.
2. Вычли смещение 127, чтобы получить реальный показатель степени двойки.
3. Прибавили это значение к `subnormal_bits`.

2.2. Приведение `x` к диапазону $[1, 2]$:

Здесь используются следующие битовые маски

$$\texttt{0x007FFFFF} \equiv \texttt{0000 0000 0111 1111 1111 1111 1111 1111}.$$

$$\texttt{0x3F800000} \equiv \texttt{0 01111111 00000000000000000000000}.$$

```
ix = (ix & 0x007FFFFF) | 0x3F800000;
conv.i = ix;
float reduced_x = conv.f;
```

Мы обнулили все биты кроме мантиссы. Потом записываем в поле экспоненты значение 127, что эквивалентно тому, что реальная экспонента равна 0. Итак, мы получили новое число `m`, у которого те же биты мантиссы, что и у оригинала, но с нулевой экспонентой. То есть теперь `m` всегда лежит в диапазоне [1.0, 2.0).

2.3. Редукция диапазона:

Для минимизации погрешности аппроксимации используется редукция диапазона мантиссы. Вместо работы с исходным интервалом [1, 2), мантисса `m` приводится к отрезку

$$\left[\frac{1}{\sqrt{2}},\, \sqrt{2}\right] \approx [0.707,\, 1.414].$$

Выбор значения $\sqrt{2}$ в качестве особой точки обоснован в классической литературе по численным методам [1]. Эта точка является средним геометрическим границ интервала [1, 2], что позволяет симметрично минимизировать максимальное отклонение аргумента от единицы:

- Если $m \in [1,\, \sqrt{2}]$, то $f = m - 1 \in [0,\, 0.414]$.
- Если $m \in \left(\sqrt{2},\, 2\right)$, то после деления на 2 мы получаем $m' \in [0.707,\, 1]$, следовательно $f = m' - 1 \in [-0.293,\, 0]$.

Таким образом, максимальное абсолютное значение $|f|$ не превышает ≈ 0.414, что значительно улучшает сходимость полинома по сравнению с несмещенным интервалом.

```
if (m > 1.41421356f) {
  m *= 0.5f;
  ++subnormal_bits;
}

double f = (double)m - 1.0;
```

3. Аппроксимация

На данном этапе выполняется вычисление логарифма от редуцированной мантиссы в форме $\ln(1 + f)$. Прямое использование ряда Тейлора для этой цели неэффективно, так как он обеспечивает высокую точность только в бесконечно малой окрестности нуля и требует большого количества членов для достижения точности в `3.5 ULP` на краях интервала редукции.

Вместо этого используется метод, реализованный в библиотеке `fdlibm` [2], основанный на переходе к вспомогательной переменной `s`:

$$s = \frac{f}{2 + f}.$$

Аппроксимация строится на разложении:

$$\ln(1 + f) \approx f - \frac{f^2}{2} + s \cdot (f^2 + R),$$

где значение R вычисляется для переменной $z = s^2$ по схеме Горнера для полинома по четным степеням:

$$R = z \cdot \Big(\mathrm{Lg}_1 + z \cdot \big(\mathrm{Lg}_2 + z \cdot (\mathrm{Lg}_3 + \ldots + z \cdot \mathrm{Lg}_7)\big)\Big)$$

Коэффициенты Lg_1, ..., Lg_7 из исходного кода `fdlibm` (файл `e_logf.c`) представлены в таблице:

Имя	Значение
Lg1	0x3F2AAAAB
Lg2	0x3ECCCCCD
Lg3	0x3E924925
Lg4	0x3E638E29
Lg5	0x3E3A3325
Lg6	0x3E1CD04F
Lg7	0x3E178897

Использование типа `double` для промежуточных вычислений полинома позволит сохранить точность мантиссы `float` и гарантировать, что ошибка аппроксимации будет значительно ниже установленного порога в `3.5 ULP`.

```
static const float Lg1 = 6.6666668653e-01f;    /* 0x3F2AAAAB */
static const float Lg2 = 4.0000000596e-01f;    /* 0x3ECCCCCD */
static const float Lg3 = 2.8571429849e-01f;    /* 0x3E924925 */
static const float Lg4 = 2.2222198546e-01f;    /* 0x3E638E29 */
static const float Lg5 = 1.8183572590e-01f;    /* 0x3E3A3325 */
static const float Lg6 = 1.5313838422e-01f;    /* 0x3E1CD04F */
static const float Lg7 = 1.4798198640e-01f;    /* 0x3E178897 */

double df = (double)f;
double s = df / (2.0 + df);
double z = s * s;
double R = z * (Lg1 + z * (Lg2 + z * (Lg3 + z * (Lg4 + z * (Lg5 + z * (Lg6 + z * Lg7))))));
```

4. Реконструкция

На финальном этапе вычисляется итоговое значение логарифма по формуле:

$$\ln(x) = k\ln(2) + \ln(1+f),$$

где k — суммарная экспонента с учетом субнормальности `x`, а $\ln(1+f)$ — значение, полученное на этапе аппроксимации.

4.1. Редукция Коди–Уэйта: Для сохранения точности в пределах `3.5 ULP` используется разделение константы $\ln(2)$ на две части: `ln2_hi` и `ln2_lo`.

- `ln2_hi` содержит старшие 24 бита мантиссы. При умножении на целое число k результат остается точно представимым во `float` без потери младших бит.

- `ln2_lo` хранит остаточную часть иррационального числа $\ln(2)$, которая учитывается при сложении с дробной частью логарифма.

4.2. Минимизация ошибки поглощения:

При сложении чисел с плавающей точкой разных порядков возникает ошибка поглощения — младшие разряды меньшего числа могут быть отброшены. Чтобы этого избежать, в коде реализован порядок суммирования от меньших величин к большим:

```
static const float ln2_hi = 6.9313812256e-01f; /* 0x3f317180 */
static const float ln2_lo = 9.0580006145e-06f; /* 0x3717f7d1 */

double hfsq = 0.5 * df * df;
double res = (double)subnormal_bits * ln2_hi +
             ((double)subnormal_bits * ln2_lo + (s * (hfsq + R) + (df - hfsq)));
```

С реализацией `ln(x)` подробно можно ознакомиться в `./logf`$_s calar.c$.

Список литературы

[1] Cody, W. J., Waite, W. *Software Manual for the Elementary Functions*. -- Prentice-Hall, 1980. -- 269 p.

[2] Sun Microsystems. *FDLIBM (Freely Distributable LIBM), file e_logf.c*. [Электронный ресурс]. URL: https://github.com/freemint/fdlibm/blob/master/e_logf.c (Дата обращения: 21.04.2026).